

Missing Provenance at Resolution Time: A Consumer-Side Study of How Much npm Version Choice Can Recover

Aakash G S

Department of Computer Science and Business Systems SRM Institute of Science and Technology Chennai, India ag5901@srmist.edu.in

Hariharan J P

Department of Computer Science and Business Systems SRM Institute of Science and Technology Chennai, India hj0956@srmist.edu.in

Jaswanth Raj

Department of Computer Science and Business Systems SRM Institute of Science and Technology Chennai, India jj3494@srmist.edu.in

Abstract—npm can attach build provenance to published package versions, and tools now let projects require it. Recent release-side measurement asks how each release reached the registry; we study the other end of the supply chain, the consumer at resolution time. We ask a practical question: when a resolved dependency lacks provenance, can a project recover it by choosing a different version inside the range it already declares, and what does that cost? On 160 randomly sampled, commit-pinned open-source repositories (262,066 resolved dependency edges) we measure a funnel with attrition at every stage. 84.80% of edges lack provenance; only 8.05% of those have an in-range provenance-bearing version within a minor/patch step of what is resolved; and only 28.51% of those (repository-clustered 95% CI [23.8, 33.3]) are accepted by npm’s own resolver without changing any other package. End to end, 2.30% of deficient edges are recoverable this way. Once a candidate passes the resolver, however, switching to it caused no attributable failure in installation, lifecycle scripts, peer dependencies, signature audit, or the repository’s own tests in any of 110 isolated experiments across 31 repositories with reproducible test suites. The limiting factor is the availability of provenance-bearing versions and their acceptance by the resolver, not breakage. All data, pre-registration tags and analysis code are public.

Index Terms—software supply chain, npm, build provenance, dependency resolution, consumer-side, empirical study

I. INTRODUCTION

Software supply-chain attacks on package registries have made the origin of a package version a first-class question. npm provenance records, as a signed attestation, the source repository and build that produced a published version [1]. Provenance is *origin evidence*; it is not a statement that a package is safe, and we never use it as one.

Most measurement of this evidence is *producer-side*: it looks at releases as they are published. Santos-Grueiro [2] compares each release with its immediate predecessor across publisher, repository, workflow, provenance and signing evidence in five registries, and turns public release-path discontinuities, including provenance disappearance, into a release-time review queue. That work deliberately sets dependency

graphs aside as describing consumption rather than production, so it does not ask what a *consumer* can do. A project does not choose releases one at a time; it declares ranges and lets a resolver pick a whole tree. This paper is a *consumer-side, resolution-time* study. Our unit is a resolved dependency edge in a real application lockfile, and our question is how much origin evidence that project can obtain through the version choices its own declared ranges and npm’s resolver allow.

Registries and package managers are beginning to let consumers require such evidence. A project that enables such a policy will find that most of its resolved dependencies lack provenance. The natural remedy is version choice: pick, within the semantic-versioning range each dependency already declares, a version that carries provenance. Whether this works in practice depends on three things that are easy to assume and hard to know: whether such versions exist, whether npm’s real resolver accepts them without disturbing the rest of the tree, and whether the project still installs and passes its tests.

We measure all three. Our contributions are:

- A consumer-side, resolution-time view of provenance that complements release-side measurement [2]: we measure origin evidence where a project actually acquires it, on resolved lockfile edges, under its own declared ranges and npm’s real resolver.
- A funnel measurement of provenance recoverability on two corpora (45 and 160 repositories), separating availability, resolver acceptance, and behaviour, with edge- and repository-weighted estimates and repository-clustered confidence intervals.
- An isolated execution harness (disposable containers, an allowlisting network proxy, a verified network self-test) that runs real installs, lifecycle scripts and test suites of untrusted packages for both the original and the substituted dependency tree.
- Evidence that the binding constraint is availability and resolution: the resolver rejects most screening-positive

candidates, lower provenance-bearing versions almost never help, and resolver-accepted substitutions showed no attributable behavioural failures.

- A public, pre-registered artifact in which every number in this paper is recomputed from raw data by a test.

II. RESEARCH QUESTIONS

RQ1 How many resolved dependency edges lack provenance? **RQ2** How many deficient edges have an in-range provenance-bearing candidate, and how many does npm’s resolver accept (single best candidate and any candidate)? **RQ3** Among resolver-accepted substitutions, how many keep the project installable, its lifecycle scripts and peer dependencies intact, its audit signature status, and its own tests passing? **RQ4** How sensitive are these answers to the definition of “without disturbing the tree”?

III. METHOD

A. Corpora

Phase 1 covers 45 well-known JavaScript projects with a committed lockfile (69,692 edges), selected in an earlier pilot. *scale_v1* is a stratified random sample designed and frozen in version control before any data were fetched: from a frame of public GitHub repositories in JavaScript or TypeScript with at least 1,000 stars, active in the last six months, we drew 20 repositories in each of eight strata (four star bands $\times$ created before/after 2018) that have a root `package-lock.json` (lockfile version ≥ 2). Each repository is pinned to the commit it was sampled at.

B. Classification (RQ1, RQ2)

We parse every lockfile into dependency edges (requester, declared range, resolved version). A version has provenance iff its registry metadata carries `dist.attestations`. An edge is *deficient* if its resolved version lacks provenance and *screening-positive* if the highest provenance-bearing version satisfying the declared range differs from the resolved one by at most a minor or patch step. All range logic uses npm’s own `semver` library.

C. Resolver confirmation, Layer 1 (RQ2, RQ4)

For each screening-positive edge we force the candidate with an `overrides` entry (or by editing the direct dependency when the package is a root dependency, which `overrides` rejects) and re-resolve with `npm install --package-lock-only --ignore-scripts`, npm 10.9.7. A candidate is *OK* if it is applied and no other top-level package changes version; otherwise *RIPPLE*, or *PEER_CONFLICT* on `ERESOLVE`. Nothing executes. A pre-registered follow-up experiment tried up to five lower provenance-bearing in-range versions for every non-OK edge.

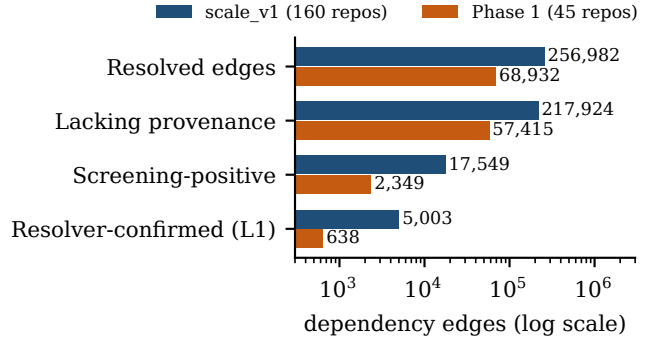


Fig. 1. Attrition from resolved edges to resolver-confirmed recoverable edges (log scale) in both corpora.

D. Behaviour, Layer 3 (RQ3)

Untrusted package code runs only in a disposable container per arm: no Linux capabilities, read-only root filesystem, non-root user, memory, CPU and process limits, the repository snapshot mounted read-only, and an internal network whose only exit is a proxy that admits the npm registry and Sigstore hosts and refuses all others. A network self-test (allowed hosts reachable, others refused and logged, direct egress impossible) must pass before any experiment. For each experiment we run the unmodified repository (B0) and the substituted one (PDR) through install, lifecycle scripts, a peer check, npm audit signatures, and the repository’s own tests, twice per arm. The PDR arm restores the original `package.json` before testing so the repository’s own linters do not react to our edit. A failure is *attributable* only if B0 passed the same stage. In *scale_v1*, a baseline screen first ran B0 alone and only repositories whose own tests passed twice proceeded.

E. Statistics and pre-registration

Edges within a repository are not independent. Every rate is reported edge-weighted and repository-weighted, each with a 95% repository-clustered percentile bootstrap (5,000 resamples, fixed seed); repository-level proportions use Wilson intervals. Designs, cohorts and analysis scripts were committed and tagged before the runs they govern; changes were made only as dated amendments with reasons stated before outcomes.

IV. RESULTS

A. RQ1: Most resolved edges lack provenance

In *scale_v1*, 84.80% of known edges lack provenance (CI [82.7, 87.0]; repository-weighted 83.4%, [80.8, 85.8]). Phase 1 gives 83.29% ([79.3, 87.0]). Unknown status is below 2% in both corpora.

B. RQ2: Few candidates exist, and the resolver rejects most

Only 8.05% of deficient edges in *scale_v1* have a screening-positive candidate ([6.7, 9.4]; repository-weighted 5.8%), and 4.09% in Phase 1. Forcing the candidate through npm’s resolver confirms 28.51% of them in *scale_v1* ([23.8, 33.3];

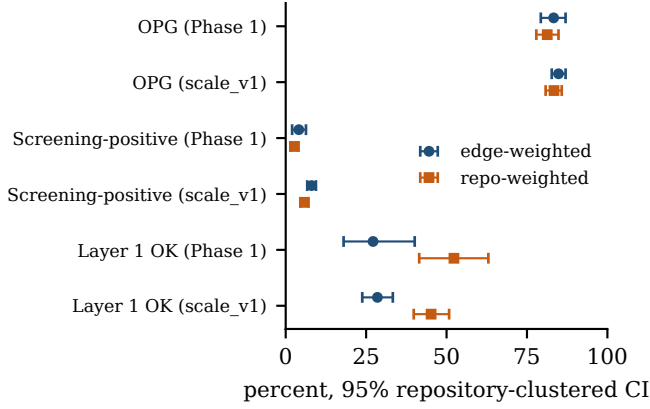


Fig. 2. Edge- and repository-weighted rates with 95% repository-clustered CIs. Screening-positive is among deficient edges; Layer 1 OK is among screening-positive edges.

TABLE I
LAYER 1 OUTCOMES (SCREENING-POSITIVE EDGES).

Layer 1 outcome	Phase 1	scale_v1
OK	638	5,003
RIPPLE	1,656	10,860
PEER_CONFLICT	55	1,573
RESOLUTION_FAIL	0	113
Total	2,349	17,549

repository-weighted 45.2%, [39.8, 50.8]) and 27.16% in Phase 1 ([18.0, 40.1]; repository-weighted 52.3%). Table I shows that rejection is overwhelmingly a *ripple*: the candidate applies, but other packages move. End to end, 2.30% of deficient edges in scale_v1 are recoverable by version choice without disturbing the tree. Edge- and repository-weighted figures diverge; in Phase 1 this is because a few repositories contribute many near-identical candidates (Fig. 2).

Trying lower versions barely helps. For the 1,711 Phase 1 edges whose best candidate failed, up to five lower provenance-bearing in-range versions recovered 12; the exists-any rate is 27.67% ([18.2, 41.5]) against 27.16% for the single best candidate. 524 of those edges had no lower candidate at all, and 384 could not be evaluated on the analysis host (reported as a bound).

C. RQ3: Accepted substitutions behaved like the baseline

In scale_v1, 31 of 91 repositories (34.1%, Wilson [25.2, 44.3]) had a test suite that installed and passed twice in the isolated worker. Across their 110 experiments, no failure at any stage was attributable to the substitution: paired failure deltas were zero for installation (0 of 110 pairs), lifecycle scripts (0) and tests (0 of 108). The 12 non-OK lifecycle outcomes occurred identically in the unmodified baseline (blocked post-install downloads, a vendored binary build). With zero events, a rule-of-three upper bound is about 2.8% per experiment and 10% per repository. Audit signature status never got worse, and 99 of 110 substitutions increased the number of verified

TABLE II
LAYER 3 OUTCOMES. MP = MICRO-PILOT ROUNDS (ENVIRONMENT FIXES BETWEEN ROUNDS WERE BASELINE-ONLY AND PRE-REGISTERED).
†SCALE_V1 SCREENED BASELINES FIRST AND RAN ONLY TESTABLE REPOSITORIES.

Outcome	MP v1	MP v2	MP v3	scale_v1
OK	9	8	12	97
TEST_FAIL	12	12	6	1
TEST_FLAKY	1	0	0	0
TIMEOUT	0	2	4	0
LIFECYCLE_FAIL	0	0	0	12
INSTALL_FAIL	1	1	1	0
Invalid baselines	13	13	11	60 of 91 repos†
Experiments	23	23	23	110

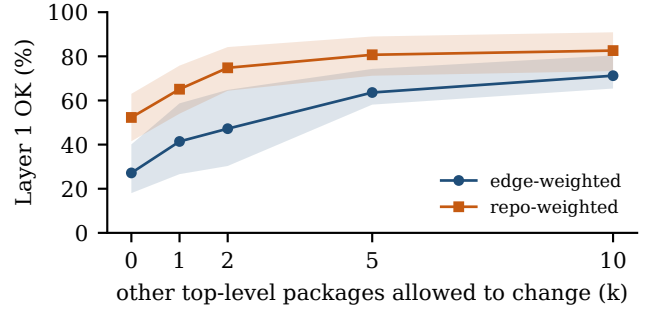


Fig. 3. Phase 1 Layer 1 OK rate when up to k other top-level packages may change version (sensitivity analysis; $k = 0$ is the frozen definition).

attestations, as intended. Table II also reports the three rounds of a pre-registered 23-experiment micro-pilot that preceded the scale-up; its final round gave 12 OK of 12 judgeable experiments, with 11 of 23 baselines unusable.

D. RQ4: The definition of “undisturbed” matters

Layer 1’s OK allows zero other packages to change. Allowing one raises the Phase 1 rate from 27.16% to 41.42% (repository-weighted 65.1%); allowing five raises it to 63.60% (80.7%) (Fig. 3). Our headline figures use the strictest definition, so they are lower bounds on what a project willing to accept small side effects could recover.

V. DISCUSSION

The funnel locates the problem. Provenance is missing for most edges, and for over nine in ten deficient edges no provenance-bearing version exists close to what is resolved. Where one exists, npm usually cannot adopt it in isolation, because adopting it moves other packages. Where npm can adopt it, we observed no behavioural cost. For maintainers this means enforcing provenance today mostly fails for lack of supply, not because switching breaks projects; for registry and tool designers it means the leverage is upstream (publishers adopting provenance) and in resolution (letting a resolver optimise for provenance rather than forcing single candidates). A configurable solver such as MaxNPM [3] could find combinations npm’s resolver does not; we measured npm’s own

behaviour and do not claim that better solutions are impossible. The two ends of the supply chain also compose. A release-side trigger such as provenance disappearance [2] tells a reviewer that a release path changed; at resolution time the same event appears as a deficient edge, and our funnel says whether the consuming project could have stayed on a provenance-bearing version without disturbing its tree. Neither view substitutes for the other.

VI. THREATS TO VALIDITY

Construct. Provenance presence is read from the registry’s current metadata, not at publish time. Tests measure what each project chose to test. *Internal.* Layer 3 covers only repositories whose own tests were reproducible in a locked-down worker (34.1% of eligible ones); projects needing network services, browsers or specific toolchains are not represented. Single-run test outcomes can be unstable [4]; we ran every arm twice and excluded flaky pairs. Our edit of `package.json` initially caused lint failures that looked like regressions; we found and removed this artifact before the scale-up. *External.* Both corpora are popular, actively maintained GitHub projects with committed lockfiles; results may not hold for the long tail of the registry or for other ecosystems. Provenance-bearing versions come from projects whose practices may differ systematically from others [5]. *Conclusion.* With zero observed attributable failures, Layer 3 bounds the failure rate rather than estimating it.

VII. RELATED WORK

Studies of npm updates measure how new versions propagate through semantic versioning ranges [6], how often non-major updates break clients [7], how dependency bots and their compatibility signals are used [8], [9], and how other projects’ tests can expose breaking versions [10]; BUMP provides reproducible breaking updates for Maven [11]. He, Vasilescu and Kästner simulate npm resolution counterfactually to show that pinning alone does not prevent supply-chain exposure [12], and peer-dependency conflicts can trap the resolver [13]. On origin evidence, signing adoption has been measured in four registries other than npm [14], and artifact verifiability including npm provenance [5]. Closest in subject is Santos-Grueiro’s release-authority study [2]. For 45,812 releases in npm, PyPI, Maven Central, crates.io and RubyGems (3,427 of them npm), it reconstructs a predecessor-aware record of publisher, repository, workflow, provenance, signing and mediation evidence and finds 204 policy-triggering release-path discontinuities that form a bounded review queue. Its unit is a package release compared with the previous release of the same package, from a purposefully sampled registry cohort; its user is a reviewer deciding whether a new release needs attention. Our unit is a resolved dependency edge in a sampled application, and our user is the project that must live with npm’s resolution. The studies answer different questions and their rates are not comparable: their npm provenance visibility is a share of sampled releases, ours a share of resolved edges. They share one signal, provenance loss between

adjacent releases, which that study uses as a review trigger and which we observe downstream as deficiency. In the corpus we searched, no directly matching work was identified that measures, on real application lockfiles, whether provenance-bearing versions exist within declared ranges and how much of that apparent recovery survives npm’s resolver, installation and the project’s own tests.

VIII. CONCLUSION

Seen from the consumer at resolution time, version choice recovers provenance for only about 2.30% of deficient dependency edges in our sample, almost entirely because suitable versions are missing or the resolver cannot adopt them without side effects. When it can, the switch did not break the projects we could test. Raising provenance coverage is therefore mainly a question of publisher adoption and provenance-aware resolution.

DATA AVAILABILITY

Code, data, pre-registration tags and the claims registry are available at <https://github.com/ftakash/PDR>.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to help build the data-collection and analysis pipeline. All code, data and results were reviewed and verified by the authors.

REFERENCES

- [1] GitHub, “npm provenance general availability (github changelog),” 2023, 2023-09-26; check exact URL before submission.
- [2] I. Santos-Grueiro, “On good authority: Release-authority measurement for registry-mediated package ecosystems,” arXiv:2606.22593, 2026.
- [3] D. Pinckney, F. Cassano, A. Guha, J. Bell, M. Culp, and T. Gamblin, “Flexible and optimal dependency management via Max-SMT,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023.
- [4] V. Maksymiuk, “How reproducible are build and test outcomes of dependency updates in JavaScript/npm projects?” Bachelor’s thesis, Delft University of Technology, 2026. [Online]. Available: https://repository.tudelft.nl/file/File_5ca45497-99f6-4280-aa7a-d64b15c86ce1
- [5] O. Solarin, K. Kalu, J. C. Davis, and P. Amusuo, “Reproducibility is not enough: Artifact verifiability in decentralized-build package ecosystems,” arXiv:2608.18180, 2026.
- [6] D. Pinckney, F. Cassano, A. Guha, and J. Bell, “A large scale analysis of semantic versioning in NPM,” arXiv:2304.00394, 2023, check: published at MSR 2023 per search index.
- [7] D. Venturini, F. R. Cogo, I. Polato, M. A. Gerosa, and I. S. Wiese, “I depended on you and you broke me: An empirical study of manifesting breaking changes in client packages,” *ACM Transactions on Software Engineering and Methodology*, 2023, arXiv:2301.04563.
- [8] R. He, H. He, Y. Zhang, and M. Zhou, “Automating dependency updates in practice: An exploratory study on GitHub Dependabot,” *IEEE Transactions on Software Engineering*, 2023, arXiv:2206.07230; check volume/pages.
- [9] B. Rombaut, F. R. Cogo, and A. E. Hassan, “Leveraging the crowd for dependency management: An empirical study on the Dependabot compatibility score,” arXiv:2403.09012, 2024.
- [10] S. Mujahid, R. Abdalkareem, E. Shihab, and S. McIntosh, “Using others’ tests to avoid breaking updates,” in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020, check title: ACM DL lists “Using Others’ Tests to Identify Breaking Updates”, DOI 10.1145/3379597.3387476.
- [11] F. Reyes, Y. Gamage, G. Skoglund, B. Baudry, and M. Monperrus, “BUMP: A benchmark of reproducible breaking dependency updates,” in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, arXiv:2401.09906.

- [12] H. He, B. Vasilescu, and C. Kästner, “Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, 2025, arXiv:2502.06662.
- [13] X. Wang, M. Wang, W. Shen, and R. Chang, “Understanding and detecting peer dependency resolving loop in npm ecosystem,” arXiv:2505.12676, 2025.
- [14] T. R. Schorlemmer, K. G. Kalu, L. Chigges, K. M. Ko, E. A. Isghair, S. Bagchi, S. Torres-Arias, and J. C. Davis, “Signing in four public software package registries: Quantity, quality, and influencing factors,” in *IEEE Symposium on Security and Privacy (S&P)*, 2024, arXiv:2401.14635; check spelling of author Saurabh Bagchi (summary of the opened page read "Baghi").